

Ujian Akhir Semester

Sistem Operasi + Jaringan Komputer — Kelas Sentul

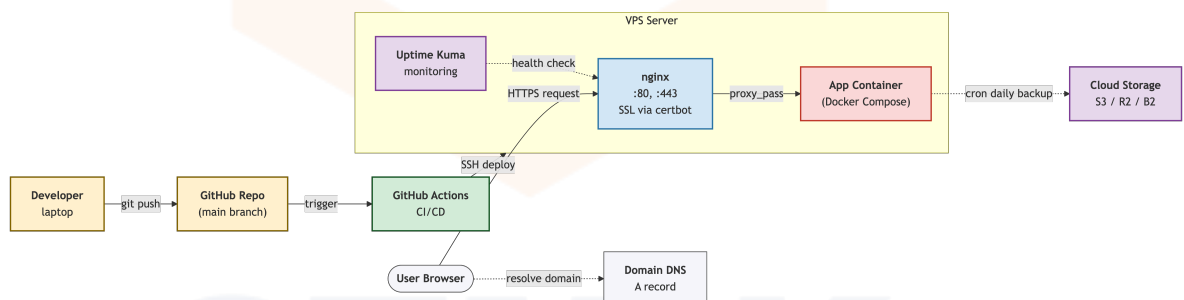
Sesi 16 · Semester Genap 2025/2026 · Combined Exam

Tentang ujian ini. Demonstrasi sistem deployment production yang dibangun selama sesi 9–15, ditambah failure recovery live. Satu sesi praktik yang menghasilkan dua nilai: **Sistem Operasi (MITI.202)** dan **Jaringan Komputer (MITI.203)**. Materi disusun selaras (loosely) dengan objectives **LPI DevOps Tools Engineer** (exam 010-160) supaya mahasiswa yang berminat dapat melanjutkan persiapan sertifikasi secara mandiri. Lihat appendix di akhir dokumen untuk pemetaan objective lengkap.

Tujuan

Memverifikasi mahasiswa berhasil mendeploy aplikasi sederhana ke production (VPS) dengan domain HTTPS + CI/CD + monitoring, dan dapat melakukan diagnosis serta recovery ketika sistem mengalami gangguan operasional.

Arsitektur Sistem Target



Alur deployment dan operasi:

1. **Developer** melakukan `git push` ke main branch di **GitHub Repo**
2. **GitHub Actions** otomatis berjalan: build image + run tests + SSH ke VPS untuk `docker compose pull && up -d`
3. **User** mengakses domain via browser: DNS resolution → IP VPS → HTTPS ke **nginx** → reverse proxy ke **App Container**
4. **nginx** terkonfigurasi sebagai reverse proxy; SSL cert dikelola oleh **certbot** (Let's Encrypt) dengan auto-renew via systemd timer atau cron
5. **Uptime Kuma** (sibling container) polling endpoint setiap menit untuk uptime monitoring
6. Cron job menjalankan backup data aplikasi setiap hari ke **Cloud Storage** eksternal (S3/R2/B2)

Format

Grup	Kelompok yang sama dengan UTS (max 3 orang)
Durasi	40 menit per kelompok

Pengamatan	Dosen utama + 1 panel observer (opsional)
Output nilai	Dua nilai independen: Sistem Operasi (100) + Jaringan (100)
Materials	Laptop kelompok dengan akses SSH ke VPS, GitHub repo aktif, browser untuk demo

Prerequisite (Dibangun Selama Sesi 9–15)

Sebelum UAS, tiap kelompok harus memiliki:

1. **Domain aktif** dari sesi 5 (still active, paid through end of semester)
2. **Aplikasi sample** (single-file ExpressJS atau static HTML, AI-generated OK) di GitHub repo public
3. **VPS deployment**: aplikasi running di VPS dengan domain → HTTPS via nginx + certbot (Let's Encrypt)
4. **CI/CD pipeline**: push ke `main` branch → GitHub Actions → auto-deploy ke VPS
5. **Monitoring**: Uptime Kuma atau setara, dashboard accessible untuk demo
6. **Logging**: aplikasi tulis log terstruktur (stdout terbaca di `journalctl` atau docker logs)
7. **Backup**: cron job harian yang backup data aplikasi ke cloud storage atau partition terpisah
8. **Dokumentasi** di README:
 - Architecture diagram
 - Runbook: restart procedure, rollback procedure, restore from backup
 - Operational notes: dimana log file, cara cek monitoring

Prerequisite Enforcement

- Sesi 15 (dry-run): dosen review semua prerequisite per kelompok
- **Lengkap**: ikut UAS full (40 menit, semua fase)
- **Parsial**: demo bagian yang siap, failure recovery tetap dijalankan kalau system di-deploy
- **Kosong**: gagal UAS untuk Jaringan, hanya bisa presentasi konsep, max score 40

Pelaksanaan per Kelompok (40 Menit)

Fase 1 — Presentasi dan Demo Working System (15 menit)

Presentasi (10 menit):

- Slide singkat: aplikasi apa, arsitektur, tech stack, lessons learned
- Diagram alur: GitHub → CI/CD → VPS → Domain (HTTPS) → User
- Pembagian peran dalam kelompok

Demo working system (5 menit):

- Browse ke `https://<domain>` dari browser → site loads correctly dengan HTTPS valid
- Tunjukkan monitoring dashboard → uptime status
- Tunjukkan struktur GitHub repo

Fase 2 — Demo CI/CD Live (5 menit)

- Dosen pilih random satu anggota kelompok untuk demo
- Anggota tersebut: edit sesuatu kecil di aplikasi (misal teks di halaman atau response API)
- Commit + push ke `main` branch dari laptop sendiri
- Buka tab GitHub Actions → tunggu workflow berjalan
- Setelah deploy selesai: refresh browser → perubahan terlihat di production
- Jelaskan apa yang baru saja terjadi (high-level pipeline flow)

Fase 3 — Failure Recovery Live (15 menit)

Ini inti UAS dan ber-bobot tertinggi.

1. Dosen menarik 1 amplop dari 8 pre-prepared scenario (lottery di depan kelompok)
2. Dosen jalankan skenario fault: SSH ke VPS, eksekusi command, atau ubah config
3. Setelah fault diintroduce, dosen catat waktu mulai
4. Kelompok harus:
 - Diagnose root cause (target: dalam 5 menit pertama)
 - Recovery — kembalikan sistem ke working state (target: dalam 15 menit total)
 - Verify sistem normal kembali (browse ke domain, cek monitoring)
5. Anggota kelompok bisa kerja bersama, tapi dosen catat siapa yang berkontribusi (untuk komponen Q&A dan individual)
6. Hard timeout 15 menit; jika belum recover, dosen jelaskan apa yang salah → partial credit untuk diagnosis

Fase 4 — Q&A Grounded (5 menit)

Pertanyaan **harus berbasis artifact** kelompok, tidak abstrak. Dosen rotasi pertanyaan ke setiap anggota.

Contoh pertanyaan grounded:

- "Buka konfigurasi nginx di `/etc/nginx/sites-available/<domain>`, tunjukkan baris `proxy_pass` yang handle reverse proxy ke aplikasi. Apa yang terjadi kalau baris itu dihapus?"
- "Buka workflow file di `.github/workflows/`. Jelaskan apa yang dilakukan step `deploy`."
- "Tunjukkan log aplikasi pada saat kemarin pukul 14:00 lewat `journalctl` atau `docker logs`. Jelaskan request pattern yang terlihat."
- "Kalau VPS hilang sekarang, langkah apa yang dilakukan? Tunjukkan runbook di README."
- "Buka file `.env` di VPS, jelaskan setiap variable. Mana yang sensitif dan kenapa?"

Pertanyaan terlarang (memancing AI MITM):

- "Apa itu CI/CD?" (definisi abstrak)
- "Jelaskan perbedaan continuous delivery vs continuous deployment" (memori konsep)
- "Best practices untuk monitoring adalah..." (textbook answer)

Katalog Skenario Failure Recovery

Dosen menyiapkan 8 amplop berisi instruksi fault. Tier difficulty dicampur supaya random draw fair.

#	Tier	Fault	Dosen Action	Expected Recovery
1	Mudah	Container app down	SSH VPS, docker compose stop app	<code>docker ps -a</code> → restart container, verify site up
2	Mudah	Reverse proxy salah port	Edit <code>proxy_pass</code> di nginx config ke port salah, <code>nginx -s reload</code>	<code>Tail /var/log/nginx/error.log</code> → fix <code>proxy_pass</code> → <code>nginx -t</code> → reload
3	Sedang	Bad deploy via CI	Dosen push commit dengan syntax error di app code	Lihat CI failure log → <code>git revert</code> di main atau fix-forward
4	Sedang	Firewall block	<code>ufw deny 443</code> di VPS	Coba browse → unreachable → <code>ufw status</code> → <code>ufw allow 443</code>
5	Sedang	Disk full	<code>dd if=/dev/zero of=/var/log/junk bs=1M count=5000</code>	<code>df -h</code> → identify junk → delete → restart affected services
6	Sulit	DNS pointing wrong	Ubah A record di registrar ke IP garbage	<code>dig <domain></code> → recognize DNS issue → fix di registrar dashboard

7	Sulit	Cert renewal broken	<code>mv /etc/letsencrypt/live/<domain> /tmp/ di VPS, nginx -s reload</code>	nginx error log shows missing cert → certbot --nginx -d <domain> untuk re-issue → reload
8	Sulit	App env var corrupted	Edit .env di VPS, ubah variable penting (DB conn string atau API key)	App log shows error → identify env var → fix .env → restart container

Pre-prepared Resources untuk Dosen

- Backup snapshot VPS sebelum mulai UAS day (revert kalau kelompok benar-benar stuck atau ada damage permanen)
- Script one-liner untuk introduce setiap fault (supaya dosen tidak butuh waktu lama)
- Cheatsheet expected recovery commands per skenario (untuk grading + ngajarin kalau timeout)
- Lembar amplop berlabel skenario 1–8 untuk lottery draw publik

Penilaian

Bobot per Komponen dan Matkul

Komponen	SO	Jar
Presentasi + arsitektur	10	15
Demo working system (HTTPS + monitoring)	10	15
Demo CI/CD live	15	10
Failure recovery — diagnosis	15	15
Failure recovery — fix	15	15
Failure recovery — narasi (komunikasi)	5	5
Q&A grounded (semua anggota berkontribusi)	15	15
Individual contribution	15	10
Total	100	100

Lulus minimum per matkul: 60.

Mengapa Bobot SO vs Jaringan Berbeda

- **SO lebih berat di CI/CD live + Q&A grounded:** testing observability skill (logs, processes, file system state), yang merupakan kompetensi sysadmin
- **Jaringan lebih berat di presentasi + demo working:** validasi arsitektur HTTPS + reverse proxy + DNS + monitoring (mostly networking concerns)
- **Failure recovery di-equal split:** kebanyakan skenario mencakup kedua domain (mis. firewall = jaringan + SO config; disk full = SO; DNS pointing wrong = jaringan)

Adjustment Fairness untuk Failure Recovery

Karena difficulty tier acak, scoring perlu disesuaikan:

- **Bobot tetap 35** (15+15+5) untuk semua tier
- **Tier mudah:** standar diagnosis dan fix lebih ketat — kelompok harus selesai dengan rapi, tidak ada exception

- **Tier sulit:** standar lebih lenient — partial credit lebih besar, dosen lebih banyak nge-guide kalau benar-benar stuck

Alternatif (ditolak demi simplicity): sliding cap di mana tier mudah max 30, tier sulit max 35.

Practice / Dry-Run di Sesi 15

- Tiap kelompok dapat 1 practice fault dari katalog (random, berbeda untuk tiap kelompok)
- Practice fault didokumentasikan dan dibahas di kelas setelah semua kelompok selesai
- Real exam fault adalah skenario yang **tidak pernah dibahas** di kelas — fresh untuk semua kelompok
- Tujuan dry-run: kelompok familiar dengan format dan tools, bukan menghafal skenario specific

Logistics

- **Jumlah kelompok:** 9 kelompok
- **Penjadwalan:** 40 menit per kelompok, sequential (dosen perlu fokus penuh per kelompok)
- **Total durasi:** $9 \times 40 = 6$ jam → butuh 1–2 hari, atau parallel session dengan multiple dosen
- **Backup plan:** jika VPS provider down atau internet bermasalah, demo dari local docker-compose dengan disclosure → max score 70 per matkul

Checklist Dosen Sebelum Hari H

- [?] Print dokumen ini untuk tiap kelompok
- [?] Print katalog failure scenario (1 set untuk dosen)
- [?] Siapkan amplop berisi skenario 1–8 untuk lottery
- [?] Backup snapshot VPS untuk setiap kelompok (kalau perlu rollback)
- [?] Verifikasi semua kelompok punya prerequisite di sesi 15
- [?] Siapkan rubric scoring sheet per kelompok per matkul (2 sheet $\times$ jumlah kelompok)
- [?] Tentukan jadwal slot per kelompok
- [?] Test sendiri 1 fault scenario end-to-end → estimasi waktu real

Pasca UAS

- Compile lessons learned dari semua kelompok → bahan refleksi semester depan
- Skenario yang paling sering gagal → indikasi materi yang perlu diperkuat di iterasi berikutnya
- Skenario yang terlalu mudah → ganti dengan yang lebih relevan
- Domain dan VPS milik mahasiswa: ingatkan untuk dipertahankan atau di-decommission dengan benar

Appendix — Pemetaan LPI DevOps Tools Engineer Objectives

Materi UAS ini menyentuh subset objective LPI DevOps Tools Engineer (exam 010-160), sertifikasi vendor-neutral untuk DevOps fundamentals (sumber: lpi.org/our-certifications/devops-overview):

Objective	Topik	UAS Coverage
701.1	Modern Software Development	Presentasi (arsitektur, decoupling)
701.2	Standard Components and Platforms	Demo (containerized app pattern)
701.3	Source Code Management (Git)	Demo CI/CD live (push, branching)
701.4	CI/CD	Demo CI/CD live + presentation
702.1	Container Usage	Demo working system (docker compose)
702.2	Container Deployment and Orchestration	Demo (compose, single-host orchestration)
702.3	Container Infrastructure	Demo (nginx ingress, image registry concept)
703.1	Cloud Deployment	Demo (VPS-as-cloud, scaling consideration di Q&A)
703.2	System Image Creation	Tidak langsung di UAS — Dockerfile dibuat di sesi 10
704.x	Configuration Management Tools (Ansible)	Gap — tidak dipakai di kurikulum ini
705.1	IT Operations and Monitoring	Demo monitoring dashboard, failure scenarios 1-2
705.2	Log Management and Analysis	Q&A grounded di logs, failure scenarios 3, 5, 8
705.3	System Troubleshooting	Failure recovery (inti UAS)

Self-Study Lanjutan untuk LPI DevOps Tools Engineer

Setelah lulus mata kuliah, mahasiswa yang ingin mengejar sertifikasi DevOps Tools Engineer dapat melengkapi gap berikut:

- **Objective 703.2:** System image creation (Packer, cloud-init bootstrapping)
- **Objective 704.1:** Ansible — playbook, inventory, roles, modules
- **Objective 704.2:** Other configuration management tools (Chef, Puppet, SaltStack) — minimal awareness
- **Objective 705.1 advanced:** Prometheus + Grafana stack (kita pakai Uptime Kuma yang lebih simple)

Resources gratis:

- learning.lpi.org — official learning materials
- docs.docker.com — Docker official docs
- docs.github.com/actions — GitHub Actions docs
- docs.ansible.com — Ansible documentation
- **“The DevOps Handbook”** by Gene Kim et al
- **“Continuous Delivery”** by Jez Humble & David Farley

Career Path Konseptual

Mahasiswa yang berniat mengejar sertifikasi LPI sebagai bukti kompetensi profesional setelah lulus:

Cert	Coverage	Cost (USD)
LPIC-1 (101+102)	Linux fundamentals + sysadmin (selaras dgn UTS)	\$200 × 2

DevOps Tools Engineer (010-160)	CI/CD + containers + cloud (selaras dgn UAS)	\$200
LPIC-2 (201+202)	Advanced sysadmin (kalau lanjut)	\$200 × 2

Tiga sertifikasi pertama (LPIC-1 + DevOps Tools Engineer) total \$600 — investasi yang reasonable untuk credential teknis yang diakui internasional.

